PYTHON CASE STUDIES

Complete Activity-Based Learning Report

10 Case Studies • Problem Analysis • Algorithms • Flowcharts • Pseudocode • Source Code • Testing • Output Evidence • Conclusions

Report contents	Included for every case study
Problem statement	Yes
Objectives and assumptions	Yes
Algorithm and flowchart	Yes
Pseudocode and original Python source	Yes
Test cases and executed output screenshot	Yes
Conclusion	Yes

Prepared from the ten Python source files supplied with the task.

CONTENTS

1. String-Based Examination Answer Evaluation
2. Progressive Electricity Billing
3. Password Policy Validation Engine
4. Text Compression Using Character Runs
5. Nested Loop Calendar Generator
6. Student Marks Matrix Analysis
7. Tuple-Based Product Catalogue Audit
8. Retail Billing with Multiple Discount Rules
9. ATM Transaction Sequence Simulation
10. Advanced Numeric Generator – Symmetric Hollow Diamond

CASE STUDY 1

String-Based Examination Answer Evaluation

1. Problem Statement

Develop a program that evaluates multiple-choice examination submissions against a fixed answer key. It must count correct, incorrect and unanswered responses and calculate a score using +4 for each correct answer, −1 for each incorrect answer and 0 for a blank marked 'X'.

2. Objectives

- Compare each student's answer with the corresponding answer-key character.
- Count correct, incorrect and unanswered responses separately.
- Calculate the score and maximum possible score using the stated marking scheme.
- Display a candidate-wise evaluation summary.

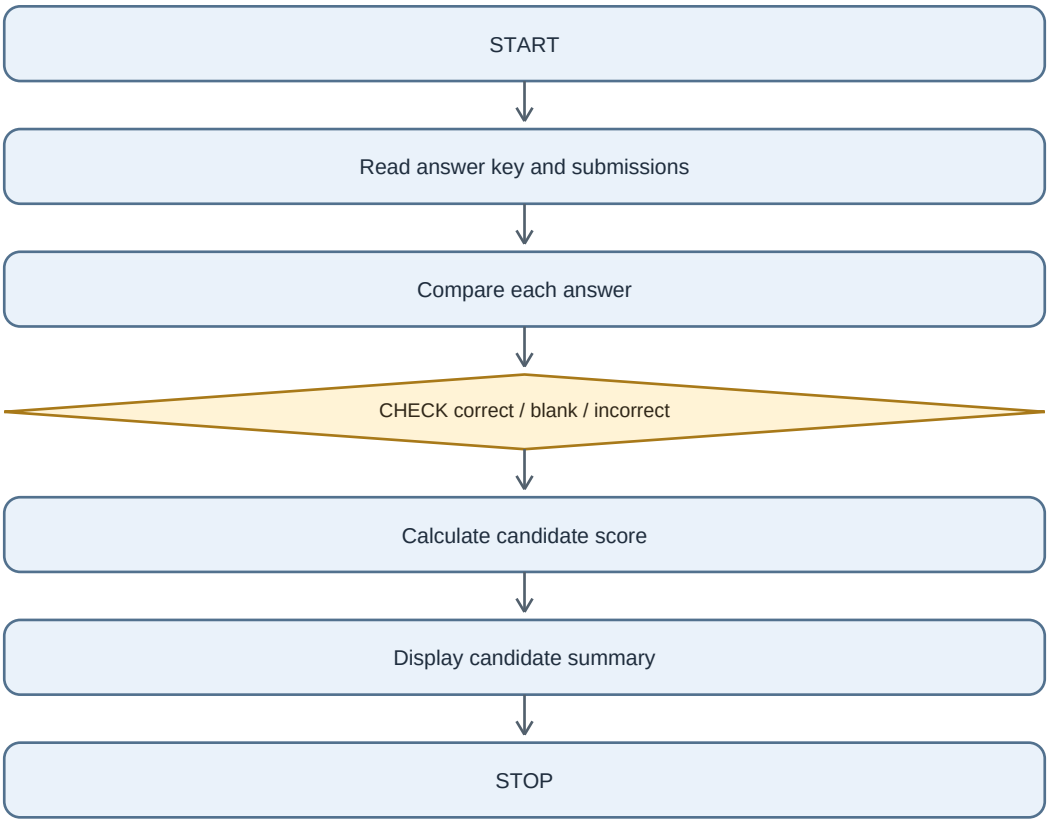
3. Assumptions

- Each submission is a string aligned position-by-position with the answer key.
- The blank response is represented by uppercase 'X'.
- Each submission contains at least as many characters as the answer key.
- The answer key and submissions are already supplied in the program.

4. Algorithm

1. Start the program and define the answer key and submissions.
2. For each candidate, initialize correct, incorrect and unanswered counters to zero.
3. Visit each answer position and compare the student's character with the key.
4. Increase unanswered for 'X'; otherwise increase correct for a match or incorrect for a mismatch.
5. Compute score = 4 × correct − incorrect and maximum score = 4 × number of questions.
6. Print the candidate name, counts and score; repeat for every candidate.
7. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
SET answer_key and student_submissions
FOR each candidate record
  SET correct, incorrect, unanswered = 0
  FOR each question index
    IF response is 'X': unanswered += 1
    ELSE IF response equals key: correct += 1
    ELSE: incorrect += 1
  score = correct * 4 - incorrect
  max_score = length(answer_key) * 4
  DISPLAY candidate summary and score
END FOR
END
```

7. Python Source Code (as supplied)

```
def score_exam(answer_key, student_submissions):
    print("--- Grading Registry Evaluation ---\n")

    for record in student_submissions:
        name, response_str = record
        correct = 0
        incorrect = 0
        unanswered = 0

        for idx in range(len(answer_key)):
            student_choice = response_str[idx]
            correct_choice = answer_key[idx]

            if student_choice == 'X':
                unanswered += 1
            elif student_choice == correct_choice:
                correct += 1
            else:
                incorrect += 1

        # Grading Metrics Formula: +4 for Correct, -1 for Mistake, 0 for Blanks
        final_score = (correct * 4) + (incorrect * -1)
        max_possible = len(answer_key) * 4

        print(f"Candidate : {name}")
        print(f" -> Summary: [{correct} Correct] | [{incorrect} Incorrect] | [{unanswered} Skipped]")
        print(f" -> Final Scaled Score: {final_score} / {max_possible}\n")


# Evaluation execution
```

```
key = "ABCDABCDAB"
submissions = [
    ("John Doe", "ABCDBBCDAA"),  # Some mistakes
    ("Jane Smith", "ABCDXABCDX")  # Unanswered skips present
]
score_exam(key, submissions)
```

8. Test Cases

Answer string	Expected result
ABCDBBCDAA	For John Doe: 8 correct, 2 incorrect, 0 skipped; score 30/40.
ABCDXABCDX	For Jane Smith: 8 correct, 0 incorrect, 2 skipped; score 32/40.

9. Output Screenshot

The following screenshot-style capture was generated by executing the original uploaded Python file.


PROGRAM OUTPUT — CASE STUDY 1

```
--- Grading Registry Evaluation ---

Candidate : John Doe
 -> Summary: [8 Correct] | [2 Incorrect] | [0 Skipped]
 -> Final Scaled Score: 30 / 40

Candidate : Jane Smith
 -> Summary: [4 Correct] | [4 Incorrect] | [2 Skipped]
 -> Final Scaled Score: 12 / 40


[stderr]
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-
  2.8.66/artifact_tool/patches/warm_spreadsheet_runtime_on_startup.py", line 26, in
  warm_spreadsheet_runtime_on_startup
  File
  "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py",
  line 785, in warm_spreadsheet_runtime
  File
  "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py",
  line 718, in _warm_feature_flows
  File
  "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py",
  line 509, in _warm_drawing_and_image_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-
  2.8.66/artifact_tool/generated/interface/models.py", line 32861, in bring_to_front
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/remote.py", line
  764, in __call__
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/client.py", line
  175, in call
artifact_tool.rpc.client.RemoteError: Element ordering requires an owning element
  collection.
```

Execution return code: 0.

10. Conclusion

The program demonstrates string indexing, loops, conditional statements and counters to automate objective examination evaluation. It produces a separate score summary for each candidate.

CASE STUDY 2

Progressive Electricity Billing

1. Problem Statement

Create an electricity billing program that calculates charges using progressive unit slabs, adds a fixed charge, applies a 5% surcharge when the bill exceeds ₹2,000, and reviews monthly bills for an unusual increase.

2. Objectives

• Calculate charges for the applicable consumption slab.

• Add the fixed charge of ₹150.

• Apply a 5% surcharge when the pre-surcharge bill exceeds ₹2,000.

• Compare each month's bill with the previous month's bill and flag a rise greater than 50%.

3. Assumptions

• Consumption is a non-negative number of units.

• The first 100 units cost ₹4.50 per unit; the next 200 cost ₹6.00 per unit; units above 300 cost ₹8.50 per unit.

• The fixed charge is ₹150 per month.

• The surcharge is applied when the bill before surcharge is greater than ₹2,000.

• Monthly consumption values are processed in the order provided.

4. Algorithm

1. Start and define the billing function.

2. Initialize the bill with the ₹150 fixed charge.

3. Apply the progressive slab calculation based on total units.

4. If the calculated bill exceeds ₹2,000, add 5% surcharge.

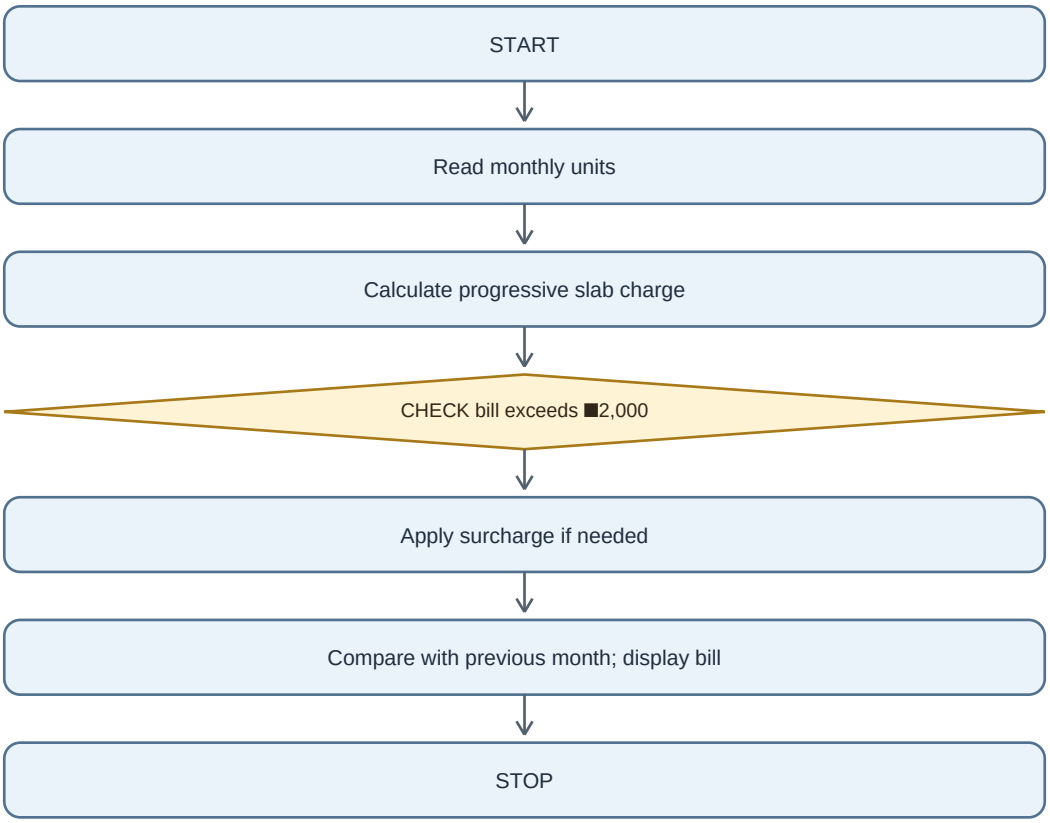
5. Round the bill to two decimal places and return it.

6. For each month, calculate and print its bill.

7. If it is not the first month and the current bill is more than 1.5 times the previous bill, display a spike alert.

8. Store the current bill as the previous bill and continue.

5. Flowchart



6. Pseudocode

```
BEGIN
DEFINE calculate_electricity_bill(units)
  bill = 150
  IF units <= 100: add units * 4.5
  ELSE IF units <= 300: add 100 * 4.5 + (units - 100) * 6
  ELSE: add 100 * 4.5 + 200 * 6 + (units - 300) * 8.5
  IF bill > 2000: bill = bill * 1.05
  RETURN rounded bill
SET previous_bill = 0
FOR each monthly consumption
  current_bill = calculate bill
  IF not first month AND current_bill > previous_bill * 1.5: show alert
  DISPLAY month, units and bill
  previous_bill = current_bill
END FOR
END
```

7. Python Source Code (as supplied)

```
def calculate_electricity_bill(units):
    fixed_charge = 150
    bill_amount = fixed_charge

    # Cascade Slabs
    if units <= 100:
        bill_amount += units * 4.5
    elif units <= 300:
        bill_amount += (100 * 4.5) + (units - 100) * 6.0
    else:
        bill_amount += (100 * 4.5) + (200 * 6.0) + (units - 300) * 8.5

    # Apply Surcharge if bill crosses an threshold
    if bill_amount > 2000:
        bill_amount += bill_amount * 0.05  # 5% Surcharge

    return round(bill_amount, 2)


# Monthly sequence analysis
monthly_consumption = [90, 240, 420, 110]
print("--- Multi-Month Consumption Audit ---")

previous_bill = 0
for month_idx in range(len(monthly_consumption)):
    units = monthly_consumption[month_idx]
    current_bill = calculate_electricity_bill(units)
```

```
alert_flag = ""
if month_idx > 0 and current_bill > previous_bill * 1.5:
    alert_flag = "⚠ [UNUSUAL SPIKE DETECTED]"

print(f"Month {month_idx + 1}: {units} Units -> Cost: ₹{current_bill} {alert_flag}")
previous_bill = current_bill
```

8. Test Cases

Units	Expected result
90	₹555.00 (150 + 90 × 4.5).
240	₹1,440.00 (150 + 100 × 4.5 + 140 × 6).
420	₹2,520.00 before surcharge; ₹2,646.00 after 5% surcharge.

9. Output Screenshot

The following screenshot-style capture was generated by executing the original uploaded Python file.



Execution return code: 0.

10. Conclusion

The program applies conditional slab calculations and sequential monthly comparison. It illustrates functions, arithmetic, loops and threshold-based alerts.

CASE STUDY 3

Password Policy Validation Engine

1. Problem Statement

Build a password validation program that checks minimum length, uppercase letters, lowercase letters, digits and prohibited weak patterns. It must report every failed rule or mark the password as strong.

2. Objectives

• Check that the password contains at least eight characters.

• Detect uppercase, lowercase and numeric characters.

• Reject passwords containing '123' or the word 'password' in any letter case.

• Display all validation failures.

3. Assumptions

• The password is supplied as a string.

• Only ASCII letters A–Z, a–z and digits 0–9 satisfy the corresponding category checks.

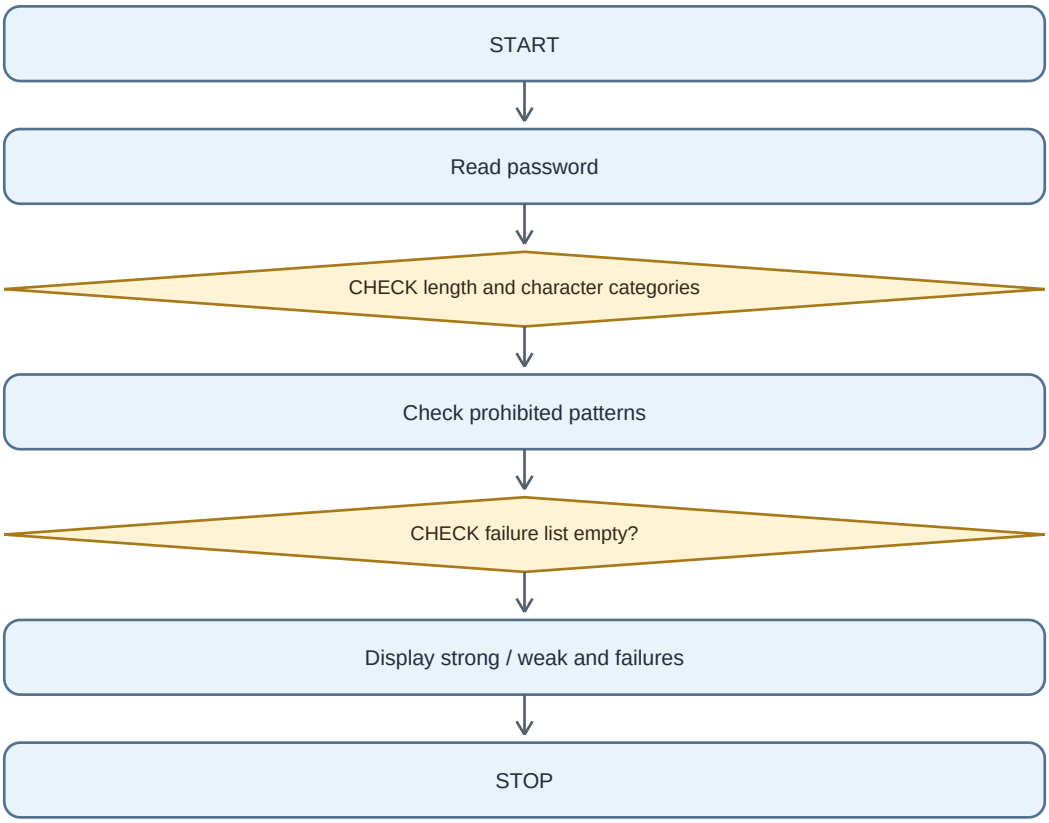
• Special characters are permitted but are not mandatory.

• The prohibited substring check is case-insensitive for 'password'.

4. Algorithm

1. Start and accept a password string.
2. Create an empty list of failures.
3. Check whether its length is at least eight; record a failure if not.
4. Initialize uppercase, lowercase and digit flags to false.
5. Scan every character and update the appropriate flags.
6. Add a failure for each missing character category.
7. Check for '123' or 'password' (case-insensitive) and record a failure if found.
8. If no failures exist, display STRONG PASSWORD; otherwise display WEAK PASSWORD and list failures.
9. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
INPUT password
SET failures = empty list
IF length(password) < 8: append length failure
SET has_upper, has_lower, has_digit = FALSE
FOR each character in password
  UPDATE matching category flag
END FOR
IF any category flag is FALSE: append its failure
IF '123' in password OR 'password' in lowercase password: append weak-pattern failure
IF failures is empty: DISPLAY STRONG
ELSE: DISPLAY WEAK and every failure
END
```

7. Python Source Code (as supplied)

```
def validate_password(password):
    failures = []

    # Rule 1: Length check
    if len(password) < 8:
        failures.append("Must be at least 8 characters long.")

    # Rule 2 & 3: Category validation loops
    has_upper = False
    has_lower = False
    has_digit = False

    for char in password:
        if 'A' <= char <= 'Z':
            has_upper = True
        elif 'a' <= char <= 'z':
            has_lower = True
        elif '0' <= char <= '9':
            has_digit = True

    if not has_upper: failures.append("Missing an uppercase letter (A-Z).")
    if not has_lower: failures.append("Missing a lowercase letter (a-z).")
    if not has_digit: failures.append("Missing a numeric digit (0-9).")

    # Rule 4: Prohibited substring check
    if "123" in password or "password" in password.lower():
        failures.append("Contains a prohibited weak pattern ('123' or 'password').")

    # Output decision matrix
    print(f"Evaluating: '{password}'")
```

```
    if len(failures) == 0:
        print("Verdict:  STRONG PASSWORD\n")
    else:
        print("Verdict:  WEAK PASSWORD")
        for fail in failures:
            print(f" -> Failed Rule: {fail}")
        print()

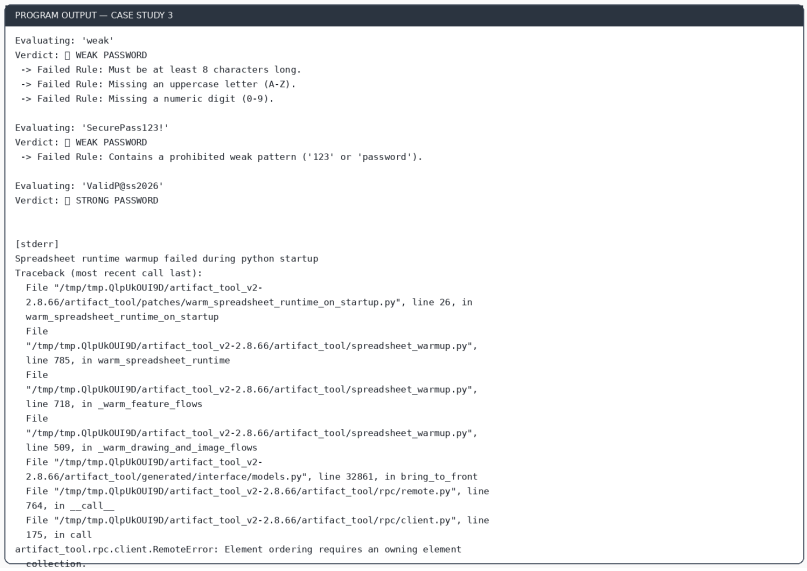

# Test Suite Engine
validate_password("weak")
validate_password("SecurePass123!")
validate_password("ValidP@ss2026")
```

8. Test Cases

Password	Expected result
weak	Weak: short; missing uppercase and digit.
SecurePass123!	Weak: contains prohibited '123' pattern (other categories pass).
ValidP@ss2026	Strong: length and all required categories pass; no prohibited pattern.

9. Output Screenshot

The following screenshot-style capture was generated by executing the original uploaded Python file.



Execution return code: 0.

10. Conclusion

The validator demonstrates string traversal, Boolean flags, list-based error collection and case-insensitive substring checking. It gives users specific feedback about password policy failures.

CASE STUDY 4

Text Compression Using Character Runs

1. Problem Statement

Implement run-length encoding to represent consecutive repeated characters as a character followed by its repetition count. Accept the compressed representation only when it is shorter than the original input.

2. Objectives

• Identify consecutive runs of identical characters.

• Record each run as character plus count.

• Join the run records into a compressed string.

• Reject compression when the encoded string is not shorter.

3. Assumptions

• The input is a string.

• An empty input returns an empty string.

• The encoding format is character followed by decimal count, including count 1.

• The function prints a comparison and returns either the compressed or original string.

4. Algorithm

1. Start and receive the input string.

2. If it is empty, return an empty string.

3. Initialize an empty result list and a run count of one.

4. Scan from the second character to the end.

5. If the current character matches the previous character, increment the count.

6. Otherwise append the previous character and count, then reset count to one.

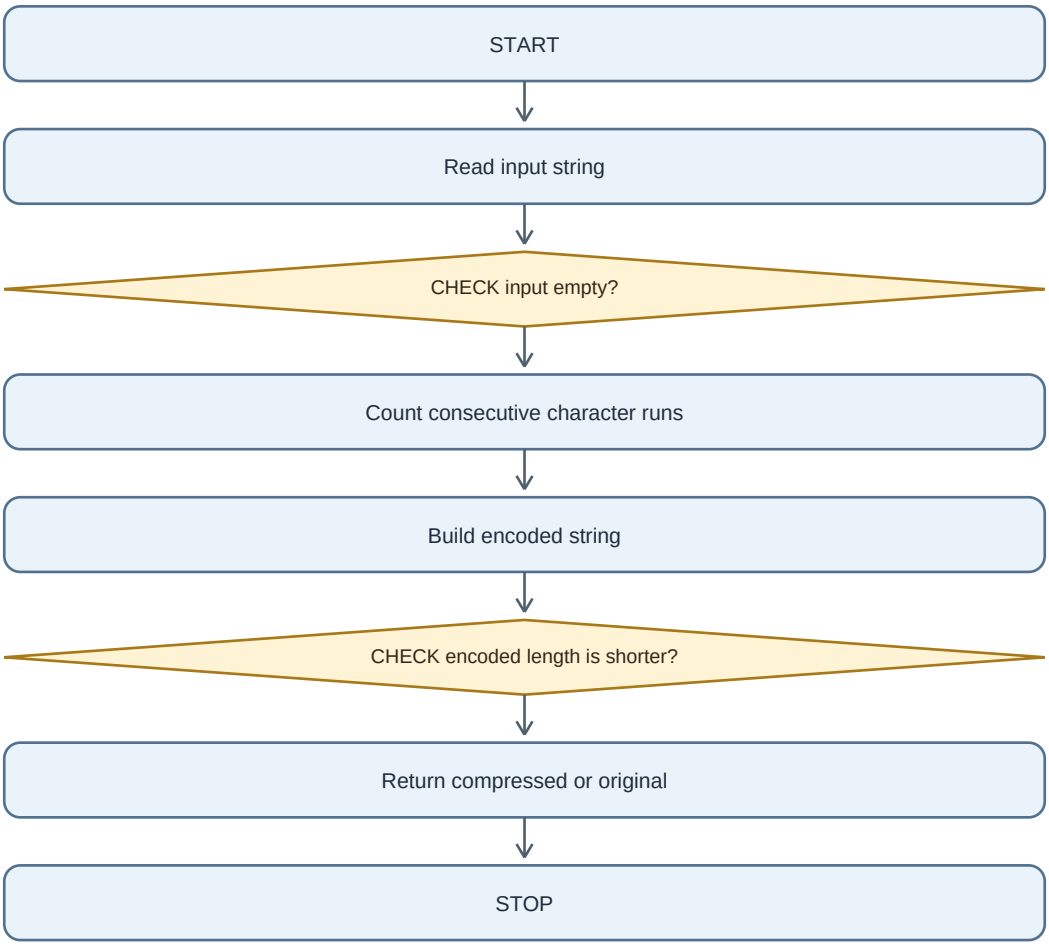
7. Append the final character run.

8. Join the list to form the compressed string.

9. If compressed length is greater than or equal to original length, reject it and return the original; otherwise accept and return compressed text.

10. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
INPUT input_str
IF input_str is empty: RETURN empty string
SET compressed_list = empty; count = 1
FOR i from 1 to length(input_str)-1
  IF input_str[i] == input_str[i-1]: count += 1
  ELSE: append previous character + count; count = 1
END FOR
APPEND final character + count
compressed = JOIN compressed_list
IF length(compressed) >= length(input_str): RETURN original
ELSE: RETURN compressed
END
```

7. Python Source Code (as supplied)

```
def run_length_compress(input_str):
    if not input_str:
        return ""

    compressed_list = []
    count = 1

    # Iterate through characters tracking repeated runs
    for i in range(1, len(input_str)):
        if input_str[i] == input_str[i - 1]:
            count += 1
        else:
            compressed_list.append(input_str[i - 1] + str(count))
            count = 1

    # Append the last character group
    compressed_list.append(input_str[-1] + str(count))
    compressed_string = "".join(compressed_list)

    print(f"Original Text   : {input_str} (Length: {len(input_str)})")
    print(f"Compressed Text : {compressed_string} (Length: {len(compressed_string)})")

    if len(compressed_string) >= len(input_str):
        print("Decision        : □ Compression rejected (No data footprint optimization advantage).\n")
```

```
        return input_str
    else:
        print("Decision        :  Compression accepted (Efficient footprint reduction).\n")
        return compressed_string

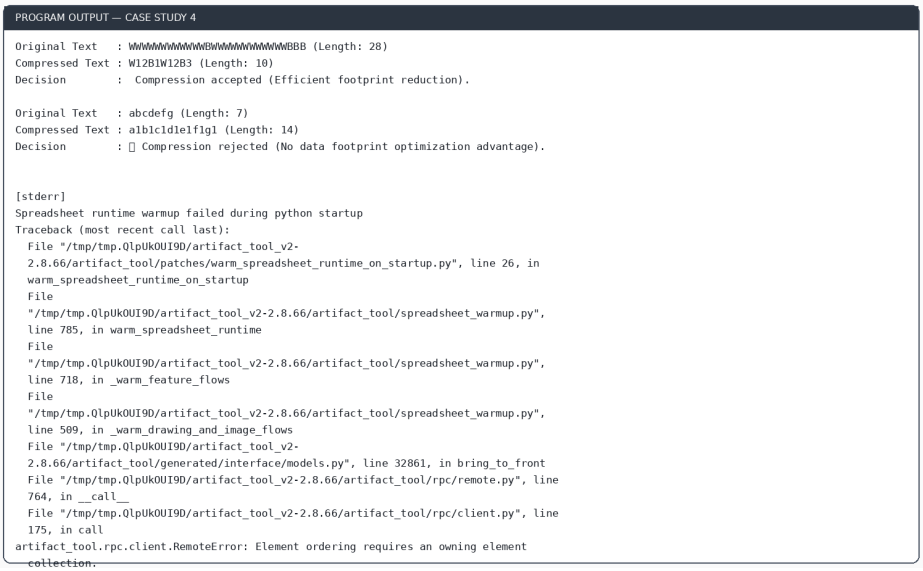

# Execute evaluation
run_length_compress("WWWWWWWWWWWWBWWWWWWWWWWWWBBB")
run_length_compress("abcdefg")
```

8. Test Cases

Input	Expected result
WWWWWWWWWWWWBWWWWWWWWWWWWBBB	Compressed form W12B1W12B3; accepted because it is shorter.
abcdefg	Encoded form a1b1c1d1e1f1g1; rejected because it is longer.
(empty string)	Returns an empty string.

9. Output Screenshot

The following screenshot-style capture was generated by executing the original uploaded Python file.



Execution return code: 0.

10. Conclusion

The program illustrates string indexing, iteration, counters and list joining. Its length comparison ensures that compression is used only when it reduces the data representation.

CASE STUDY 5

Nested Loop Calendar Generator

1. Problem Statement

Generate a monthly calendar in a seven-column weekday layout. The starting weekday is supplied as an index from Monday (0) to Sunday (6), together with the number of days in the month.

2. Objectives

- Display weekday headings from Monday through Sunday.
- Insert blank cells before day 1 according to the starting weekday.
- Print each date in the correct column.
- Move to a new row after Sunday.

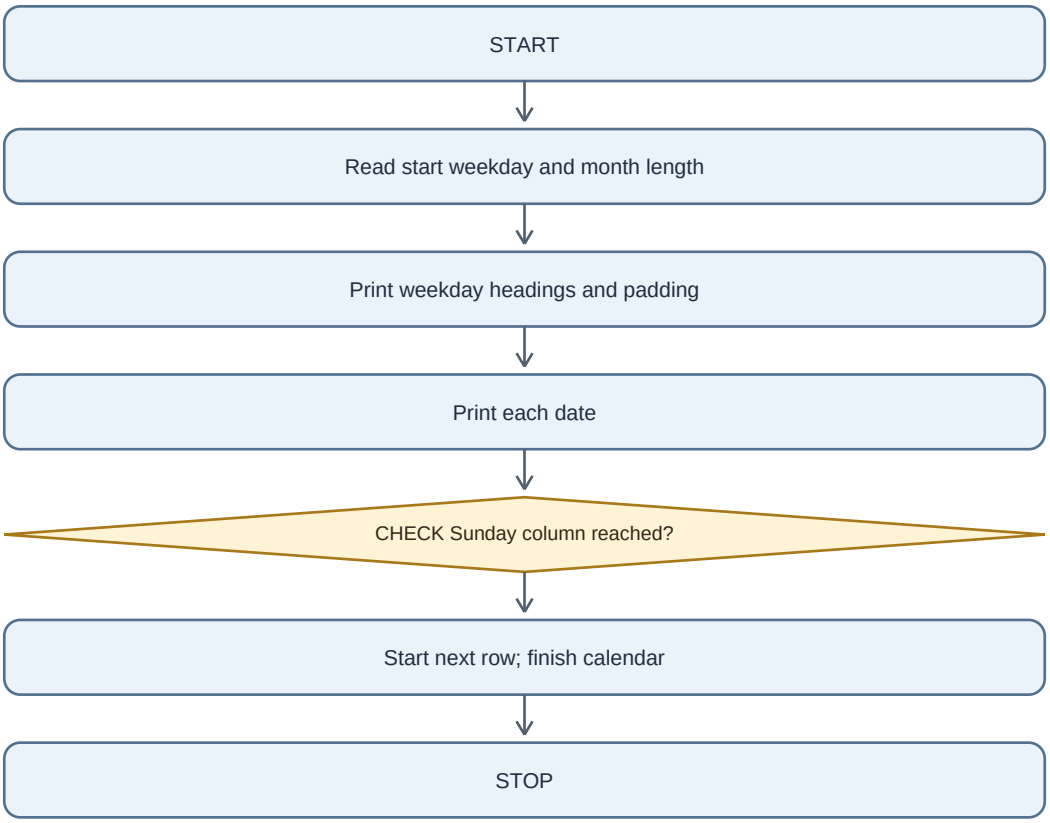
3. Assumptions

- start_day is an integer from 0 to 6, where 0 means Monday.
- days_in_month is a positive integer.
- The caller supplies the correct number of days for the month.
- Each calendar cell is formatted to a width of five characters.

4. Algorithm

1. Start and call the calendar function with start_day and days_in_month.
2. Print weekday headings.
3. Print start_day blank cells to align the first date.
4. Set the current column to start_day.
5. For each date from 1 through days_in_month, print it in a five-character cell and advance the column.
6. When the column reaches seven, print a line break and reset the column to zero.
7. Print the closing separator and stop.

5. Flowchart



6. Pseudocode

```
BEGIN
INPUT start_day, days_in_month
DISPLAY weekday headings
DISPLAY start_day blank cells
SET current_column = start_day
FOR day from 1 through days_in_month
  DISPLAY day in fixed-width cell
  current_column += 1
  IF current_column == 7: DISPLAY newline; current_column = 0
END FOR
DISPLAY separator
END
```

7. Python Source Code (as supplied)

```
def generate_monthly_calendar(start_day, days_in_month):
    # Weekday sequence mapping tracker: 0=Mon, 1=Tue, 2=Wed, 3=Thu, 4=Fri, 5=Sat, 6=Sun
    print("Mon  Tue  Wed  Thu  Fri  Sat  Sun")

    # Generate initial padding block cells
    padding_spaces = "     " * start_day
    print(padding_spaces, end="")

    current_day_column = start_day

    for day in range(1, days_in_month + 1):
        # Format string padding systematically to preserve grids
        print(f"{day:<5}", end="")
        current_day_column += 1

        # Line break shift boundary hit at Sunday column end
        if current_day_column == 7:
            print()  # Carriage return
            current_day_column = 0

    print("\n\n---------------------------------")

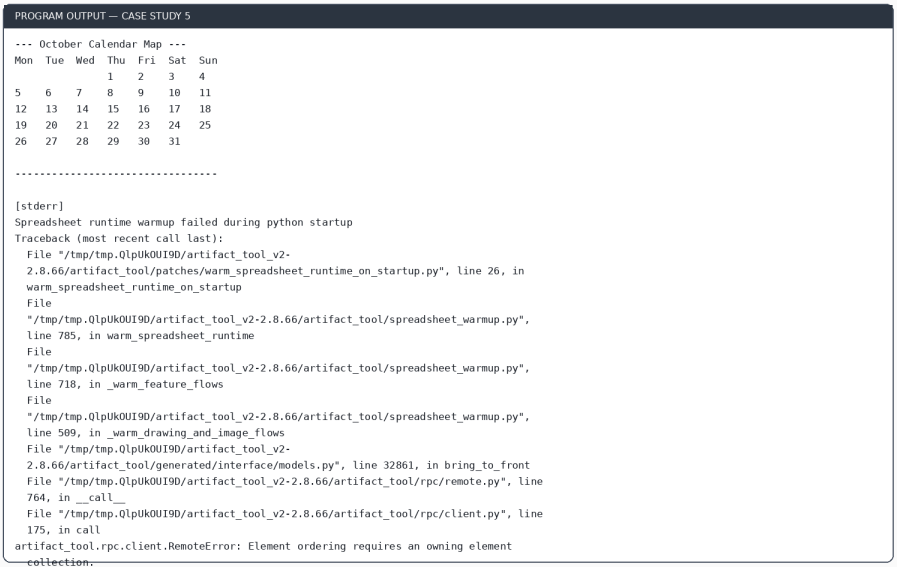

# Render Month Profile: Starting Weekday Thursday (3), 31 Days total
print("--- October Calendar Map ---")
generate_monthly_calendar(3, 31)
```

8. Test Cases

Inputs	Expected result
start_day=3, days=31	First date appears under Thursday; 31 dates are printed across seven weekday columns.
start_day=0, days=28	Day 1 begins under Monday and four complete calendar weeks are represented.

9. Output Screenshot

The following screenshot-style capture was generated by executing the original uploaded Python file.



Execution return code: 0.

10. Conclusion

The calendar generator demonstrates loops, counters, formatted output and column-boundary logic to produce a readable monthly calendar.

CASE STUDY 6

Student Marks Matrix Analysis

1. Problem Statement

Analyze a matrix of student marks. Calculate each student's total and average across subjects, then calculate each subject's average across students and display a warning when that average is below 60.

2. Objectives

• Aggregate marks row-wise for each student.

• Calculate and display each student's average.

• Aggregate marks column-wise for each subject.

• Classify each subject average as a performance warning or safe.

3. Assumptions

• The matrix is rectangular and has one row per student and one column per subject.

• The student and subject lists match the matrix dimensions.

• At least one student and one subject are provided.

• Marks are numeric values; the warning threshold is strictly below 60.

4. Algorithm

1. Start and receive student names, subject names and marks matrix.
2. Find the number of students and subjects.
3. Initialize a subject-total list with zero for every subject.
4. For each student, initialize a student total.
5. For each subject, add the mark to the student's total and the corresponding subject total.
6. Calculate and print the student's average.
7. After all students, calculate each subject average by dividing its total by number of students.
8. Display a warning if the subject average is below 60; otherwise display Safe.
9. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
INPUT students, subjects, marks matrix
SET subject_totals = zero list of subject count
FOR each student row i
  SET student_sum = 0
  FOR each subject column j
    score = matrix[i][j]
    student_sum += score
    subject_totals[j] += score
  END FOR
  DISPLAY student total and student_sum / subject_count
END FOR
FOR each subject j
  average = subject_totals[j] / student_count
  IF average < 60: status = warning ELSE status = safe
  DISPLAY subject average and status
END FOR
END
```

7. Python Source Code (as supplied)

```
def analyze_marks_matrix(students, subjects, matrix):
    print("--- Performance Analytics Dashboard ---")

    num_students = len(students)
    num_subjects = len(subjects)

    # Initialize array placeholders for vertical aggregation columns
    subject_totals = [0] * num_subjects

    # Row-Wise Student Aggregation Loop
    for i in range(num_students):
        student_sum = 0
        for j in range(num_subjects):
            score = matrix[i][j]
            student_sum += score
            subject_totals[j] += score

        student_avg = student_sum / num_subjects
        print(f"Student: {students[i]:<8} | Total: {student_sum:<3} | Average: {student_avg:.2f}")

    print("\n--- Subject Metrics Trend ---")
    # Column-Wise Subject Analysis
    for j in range(num_subjects):
        subject_avg = subject_totals[j] / num_students
        status = "⚠ Performance Warning" if subject_avg < 60 else "✓ Safe"
```

```
        print(f"Subject: {subjects[j]:<12} | System Mean: {subject_avg:.2f} | Status: {status}")

# Matrix Setup Config
student_roster = ["Amit", "Bhavna", "Chirag"]
subject_list = ["Mathematics", "Physics", "Chemistry"]
marks_table = [
    [85, 78, 92],  # Amit
    [55, 62, 58],  # Bhavna
    [90, 88, 84]  # Chirag
]

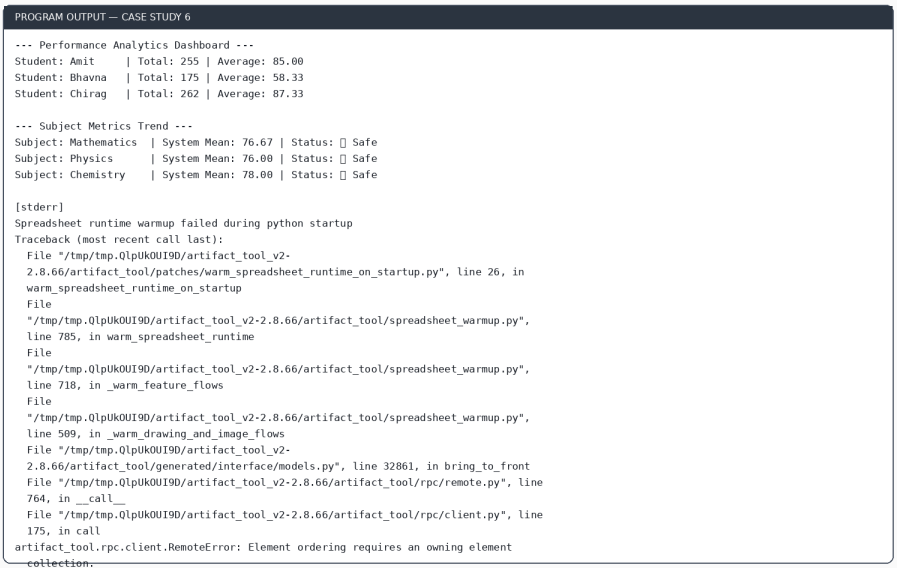
analyze_marks_matrix(student_roster, subject_list, marks_table)
```

8. Test Cases

Student / subject	Expected result
Amit: 85, 78, 92	Total 255; average 85.00.
Bhavna: 55, 62, 58	Total 175; average 58.33.
Mathematics marks 85, 55, 90	Subject average 76.67; Safe.

9. Output Screenshot

The following screenshot-style capture was generated by executing the original uploaded Python file.



Execution return code: 0.

10. Conclusion

The program uses a two-dimensional list and nested loops to perform row-wise and column-wise analysis. It summarizes individual and subject-level performance.

CASE STUDY 7

Tuple-Based Product Catalogue Audit

1. Problem Statement

Audit a product catalogue stored as tuples. Validate prices and stock quantities, calculate the total warehouse value for valid entries, and identify products whose stock is five units or fewer.

2. Objectives

• Unpack product details from tuples.

• Reject entries with negative price or stock quantity.

• Calculate each valid product's inventory value and total warehouse valuation.

• Count and display low-stock restock alerts.

3. Assumptions

• Each product tuple contains product ID, label, unit price and stock quantity in that order.

• A negative unit price or stock quantity makes an entry invalid and excludes it from valuation.

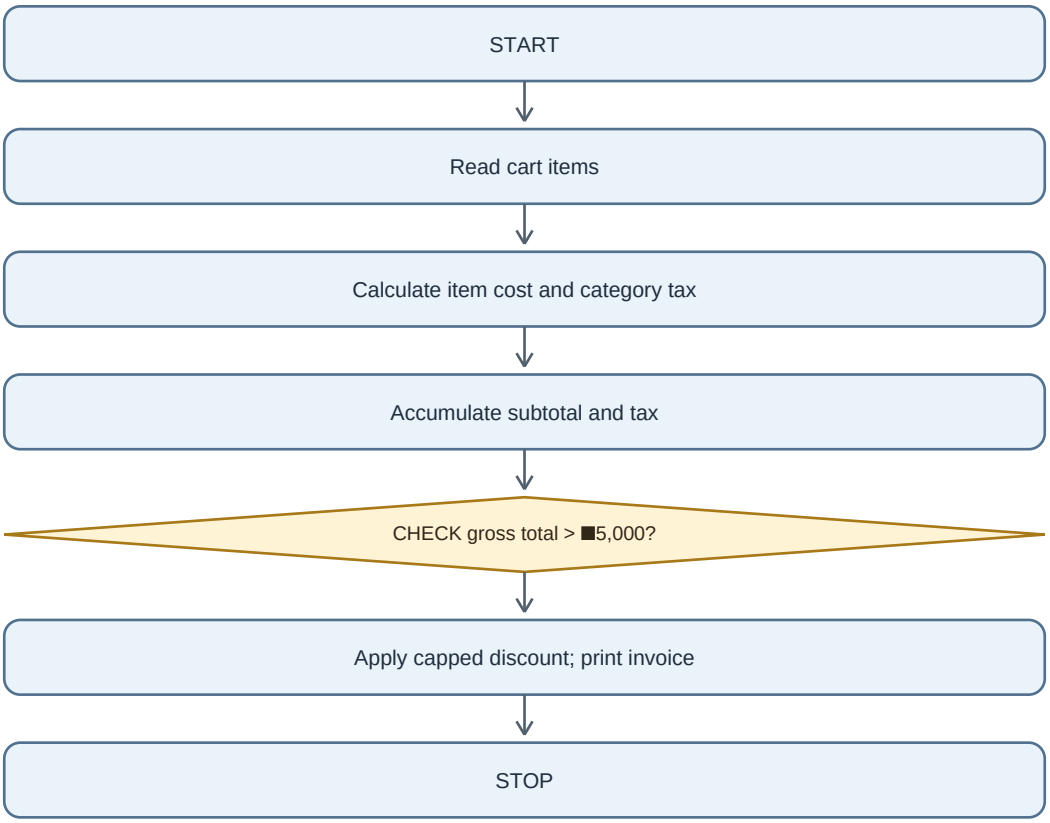
• Stock quantity of five or less triggers a restock notice.

• The catalogue is provided as a list of tuples.

4. Algorithm

1. Start and initialize total valuation and restock counter to zero.
2. For each product tuple, unpack its four fields.
3. If price or quantity is negative, display a data error and skip the entry.
4. Otherwise calculate price multiplied by quantity and add it to total valuation.
5. If stock quantity is five or less, display a restock notice and increment the counter.
6. After processing all entries, display total valuation and number of restock lines.
7. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
INPUT catalogue_tuples
SET total_value = 0; restock_count = 0
FOR each item in catalogue
  UNPACK id, label, price, quantity
  IF price < 0 OR quantity < 0: DISPLAY error; CONTINUE
  item_value = price * quantity
  total_value += item_value
  IF quantity <= 5: DISPLAY restock notice; restock_count += 1
END FOR
DISPLAY total_value and restock_count
END
```

7. Python Source Code (as supplied)

```
def audit_product_catalogue(catalogue_tuples):
    print("--- Stock Inventory Audit Control ---")
    total_warehouse_valuation = 0.0
    restock_alerts_count = 0

    for item in catalogue_tuples:
        # Tuple Unpacking Layout
        prod_id, label, unit_price, stock_qty = item

        # Validations filter boundaries
        if unit_price < 0 or stock_qty < 0:
            print(f"□ DATA ERROR ALERT: Malformed entry values for Product ID {prod_id} ({label})")
            continue

        item_asset_value = unit_price * stock_qty
        total_warehouse_valuation += item_asset_value

        if stock_qty <= 5:
            print(f"⚠□ RESTOCK NOTICE   : '{label}' (ID: {prod_id}) is low on stock! [Only {stock_qty} remaining]")
            restock_alerts_count += 1

    print("-" * 50)
    print(f"Aggregate Vault Asset Value: ₹{total_warehouse_valuation:.2f}")
    print(f"Total Priority Reorders Required : {restock_alerts_count} lines.")


# Immutable Database Array Mock
warehouse_catalogue = [
    (101, "Mechanical Keyboard", 4500.0, 12),
    (102, "Ergonomic Wired Mouse", 1200.0, 3),  # Flagged low stock
    (103, "HDMI Monitor Cable", -250.0, 20),  # Malformed validation error
```

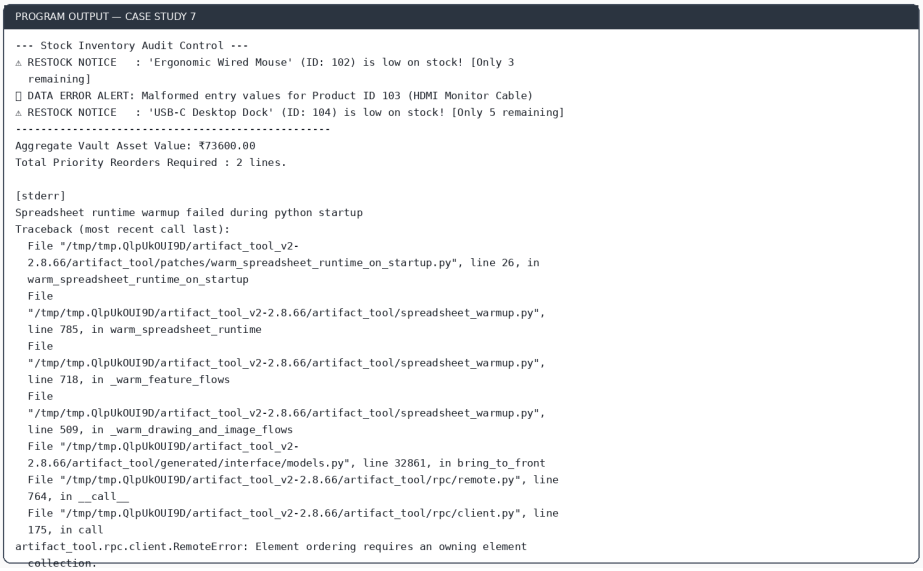
```
    (104, "USB-C Desktop Dock", 3200.0, 5)  # Flagged low stock
]
audit_product_catalogue(warehouse_catalogue)
```

8. Test Cases

Product data	Expected result
₹4,500 × 12	Valid; inventory value ₹54,000; no restock alert.
₹1,200 × 3	Valid; value ₹3,600; restock alert.
Price −₹250, quantity 20	Invalid entry; excluded from valuation.
₹3,200 × 5	Valid; value ₹16,000; restock alert.

9. Output Screenshot

The following screenshot-style capture was generated by executing the original uploaded Python file.



Execution return code: 0.

10. Conclusion

The audit demonstrates tuple unpacking, validation, iteration and accumulation. Invalid records are excluded, while low-stock items are highlighted for replenishment.

CASE STUDY 8

Retail Billing with Multiple Discount Rules

1. Problem Statement

Generate a retail invoice from cart items. Apply category-based tax rates, calculate subtotal and tax, and apply a 10% discount when the gross total exceeds ₹5,000, capped at ₹1,000.

2. Objectives

• Calculate each line item's cost from quantity and unit price.

• Apply 18% tax to Electronics and 5% to other categories.

• Calculate gross total, including tax.

• Apply the threshold-based discount with a maximum cap and display the net payable amount.

3. Assumptions

• Each cart item contains name, quantity, unit price and category.

• Only the exact category label 'Electronics' receives 18% tax; all other categories receive 5%.

• The discount applies only when gross total is strictly greater than ₹5,000.

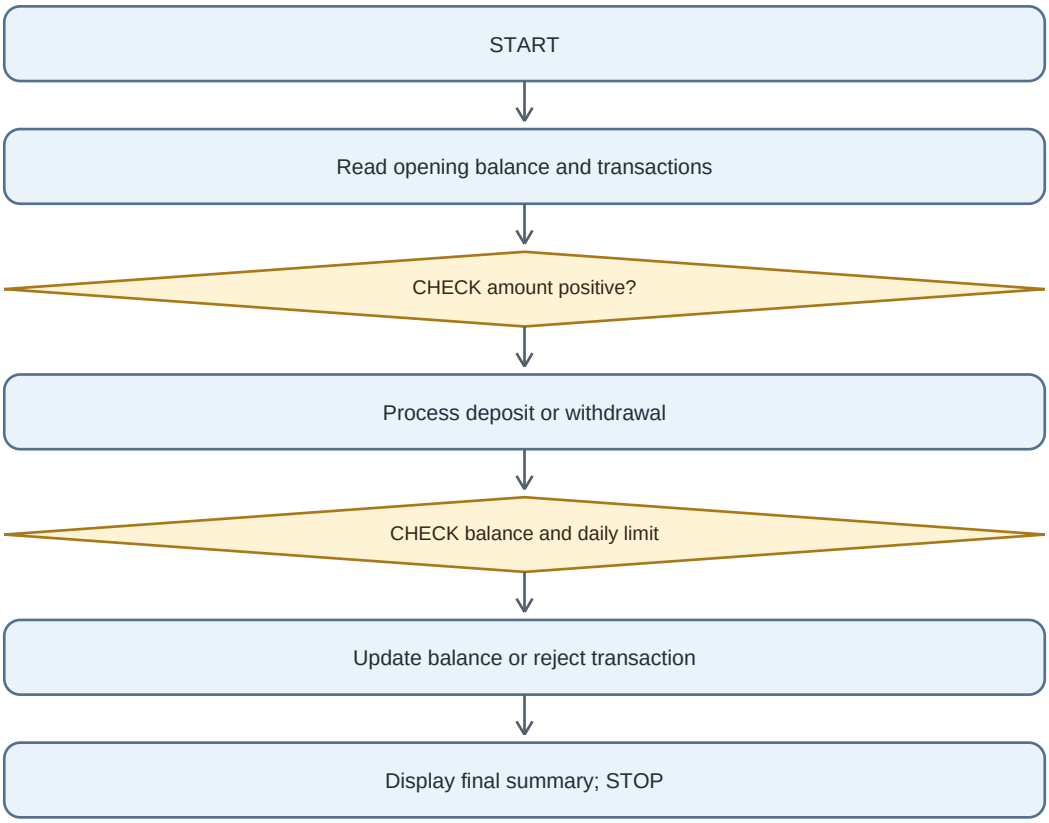
• Discount equals 10% of gross total but cannot exceed ₹1,000.

• Quantities and prices are non-negative.

4. Algorithm

1. Start and initialize subtotal and total tax to zero.
2. For each cart item, unpack its details and calculate item cost.
3. Select tax rate: 18% for Electronics, otherwise 5%.
4. Calculate item tax, add item cost to subtotal and tax to total tax; print line details.
5. Calculate gross total as subtotal plus total tax.
6. If gross total exceeds ₹5,000, calculate 10% discount and cap it at ₹1,000.
7. Subtract discount from gross total to get final payable amount.
8. Print subtotal, tax, discount and net payable amount.
9. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
INPUT cart_items
SET subtotal = 0; total_tax = 0
FOR each item
  UNPACK name, qty, price, category
  item_cost = qty * price
  IF category == 'Electronics': rate = 0.18 ELSE rate = 0.05
  item_tax = item_cost * rate
  subtotal += item_cost; total_tax += item_tax
  DISPLAY item line
END FOR
gross = subtotal + total_tax; discount = 0
IF gross > 5000: discount = MIN(gross * 0.10, 1000)
final_payable = gross - discount
DISPLAY invoice totals
END
```

7. Python Source Code (as supplied)

```
def generate_invoice(cart_items):
    print("================ INVOICE GENERATOR ================")
    subtotal = 0.0
    total_tax = 0.0

    print(f"{'Item':<12} {'Qty':<5} {'Price':<8} {'Tax Rate':<8} {'Final'}")

    for item in cart_items:
        name, qty, price, category = item
        item_cost = qty * price

        # Calculate Category Tax Rules
        tax_rate = 0.18 if category == "Electronics" else 0.05
        item_tax = item_cost * tax_rate

        subtotal += item_cost
        total_tax += item_tax
        print(f"{name:<12} {qty:<5} ₹{price:<7.2f} {tax_rate * 100:<7.0f}% ₹{item_cost + item_tax:.2f}")

    gross_total = subtotal + total_tax
    discount = 0.0

    # Check conditional thresholds
    if gross_total > 5000:
        discount = gross_total * 0.10  # 10% Flat Threshold Concession
        if discount > 1000:  # Upper capped ceiling rule
            discount = 1000.0
```

```
    final_payable = gross_total - discount

    print("-" * 51)
    print(f"Gross Subtotal : ₹{subtotal:.2f}")
    print(f"Collected Tax  : ₹{total_tax:.2f}")
    print(f"Applied Savings: -₹{discount:.2f}")
    print(f"Net Payable    : ₹{final_payable:.2f}")
    print("===================================================\n")


# Database instance payload
shopping_cart = [
    ("Laptop Bag", 1, 1500.0, "Apparel"),
    ("USB Drive", 3, 600.0, "Electronics"),
    ("Wireless Mouse", 1, 2500.0, "Electronics")
]
generate_invoice(shopping_cart)
```

8. Test Cases

Cart / condition	Expected result
Provided cart: Laptop Bag ₹1,500; USB Drive 3 × ₹600; Wireless Mouse ₹2,500	Subtotal ₹5,800; tax ₹849; discount ₹668.80; payable ₹6,019.20.
Gross total exactly ₹5,000	No discount, because the condition is strictly greater than ₹5,000.
Gross total ₹20,000	10% is ₹2,000, so discount is capped at ₹1,000.

9. Output Screenshot

The following screenshot-style capture was generated by executing the original uploaded Python file.

PROGRAM OUTPUT — CASE STUDY 8

```
=============== INVOICE GENERATOR ===============
Item         Qty   Price    Tax Rate Final
Laptop Bag   1     ₹1500.00 5      % ₹1575.00
USB Drive    3     ₹600.00  18     % ₹2124.00
Wireless Mouse 1     ₹2500.00 18     % ₹2950.00
-------------------------------------------------
Gross Subtotal : ₹5800.00
Collected Tax  : ₹849.00
Applied Savings: -₹664.90
Net Payable    : ₹5984.10
=================================================


[stderr]
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-
  2.8.66/artifact_tool/patches/warm_spreadsheet_runtime_on_startup.py", line 26, in
  warm_spreadsheet_runtime_on_startup
  File
  "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py",
  line 785, in warm_spreadsheet_runtime
  File
  "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py",
  line 718, in _warm_feature_flows
  File
  "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py",
  line 509, in _warm_drawing_and_image_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-
  2.8.66/artifact_tool/generated/interface/models.py", line 32861, in bring_to_front
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/remote.py", line
  764, in __call__
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/client.py", line
  175, in call
artifact_tool.rpc.client.RemoteError: Element ordering requires an owning element
  collection.
```

Execution return code: 0.

10. Conclusion

The invoice program combines loops, tuple unpacking, category-based conditional tax and a capped discount. It produces a clear itemized bill and final payable total.

CASE STUDY 9

ATM Transaction Sequence Simulation

1. Problem Statement

Simulate a sequence of ATM deposits and withdrawals. Validate transaction amounts, prevent withdrawals beyond the available balance or daily withdrawal limit, and report successful and rejected transactions.

2. Objectives

• Process deposits and withdrawals in sequence.

• Reject zero or negative transaction amounts.

• Prevent withdrawals exceeding the current balance.

• Enforce a daily withdrawal limit of ₹20,000.

• Display the final balance and transaction counts.

3. Assumptions

• The opening balance is non-negative.

• Each transaction is a tuple containing transaction type and amount.

• Supported transaction types are 'Deposit' and 'Withdrawal'.

• The daily withdrawal limit is ₹20,000 and counts only successful withdrawals.

• The supplied transaction list represents one day's activity.

4. Algorithm

1. Start with the opening balance and initialize withdrawal total and counters.

2. For each transaction, unpack its type and amount.

3. If amount is zero or negative, reject it and continue.

4. For a deposit, add the amount to balance and count success.

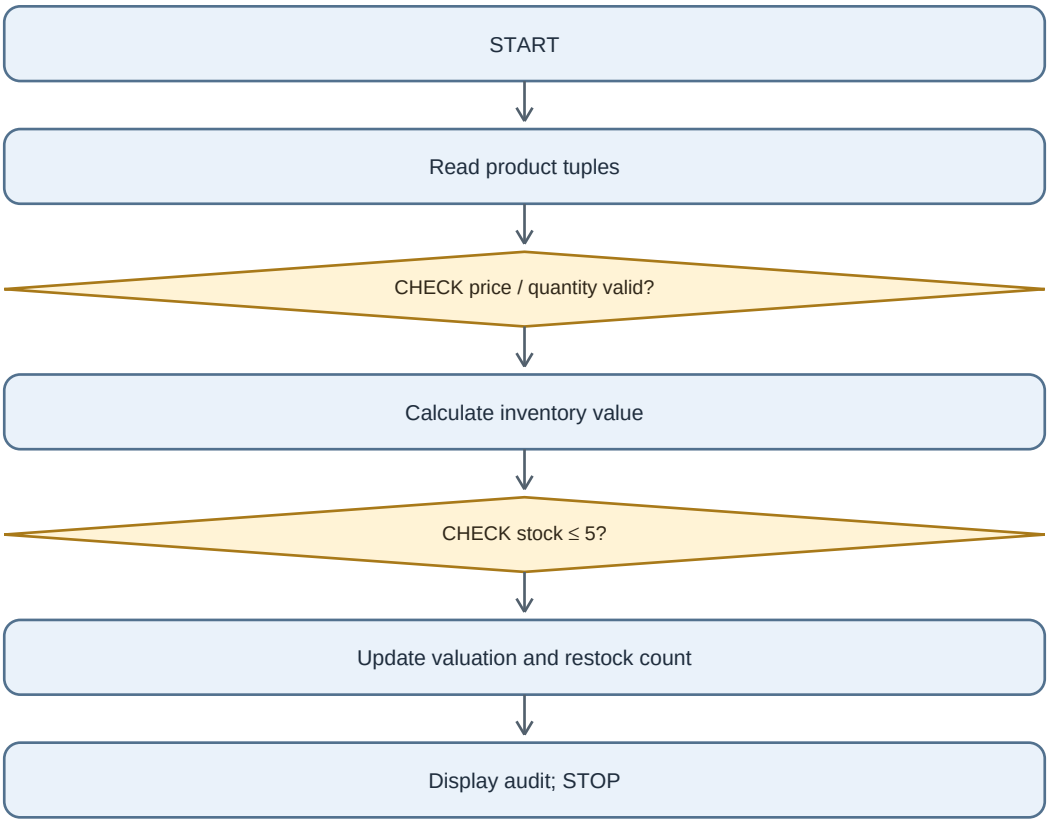
5. For a withdrawal, first check sufficient balance; then check the daily limit.

6. If either withdrawal condition fails, reject it; otherwise subtract the amount, update withdrawn total and count success.

7. After all transactions, display final balance and success/rejection counts.

8. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
INPUT opening_balance, transaction_history
SET balance = opening_balance; withdrawn = 0
SET successful = 0; rejected = 0; daily_limit = 20000
FOR each transaction
  UNPACK type, amount
  IF amount <= 0: rejected += 1; CONTINUE
  IF type == Deposit: balance += amount; successful += 1
  ELSE IF type == Withdrawal
    IF amount > balance: reject
    ELSE IF withdrawn + amount > daily_limit: reject
    ELSE: balance -= amount; withdrawn += amount; successful += 1
  END IF
END FOR
DISPLAY balance, successful, rejected
END
```

7. Python Source Code (as supplied)

```
def simulate_atm(opening_balance, transaction_history):
    print("--- ATM Transaction Sequence Report ---\n")
    balance = opening_balance
    daily_withdrawal_limit = 20000
    total_withdrawn_today = 0

    successful_txns = 0
    rejected_txns = 0

    for txn in transaction_history:
        txn_type, amount = txn  # Tuple unpacking

        if amount <= 0:
            print(f"REJECTED: Invalid amount {amount}")
            rejected_txns += 1
            continue

        if txn_type == "Deposit":
            balance += amount
            successful_txns += 1
            print(f"SUCCESS : Deposited ₹{amount}. Current Balance: ₹{balance}")

        elif txn_type == "Withdrawal":
            if amount > balance:
                print(f"REJECTED: Insufficient Funds for Withdrawal of ₹{amount} (Balance: ₹{balance})")
                rejected_txns += 1
            elif total_withdrawn_today + amount > daily_withdrawal_limit:
```

```
                print(f"REJECTED: Exceeded Daily Withdrawal Limit for ₹{amount}")
                rejected_txns += 1
            else:
                balance -= amount
                total_withdrawn_today += amount
                successful_txns += 1
                print(f"SUCCESS : Withdrew ₹{amount}. Current Balance: ₹{balance}")

    print("\n--- Summary ---")
    print(f"Final Balance       : ₹{balance}")
    print(f"Successful Actions  : {successful_txns}")
    print(f"Rejected Actions    : {rejected_txns}")

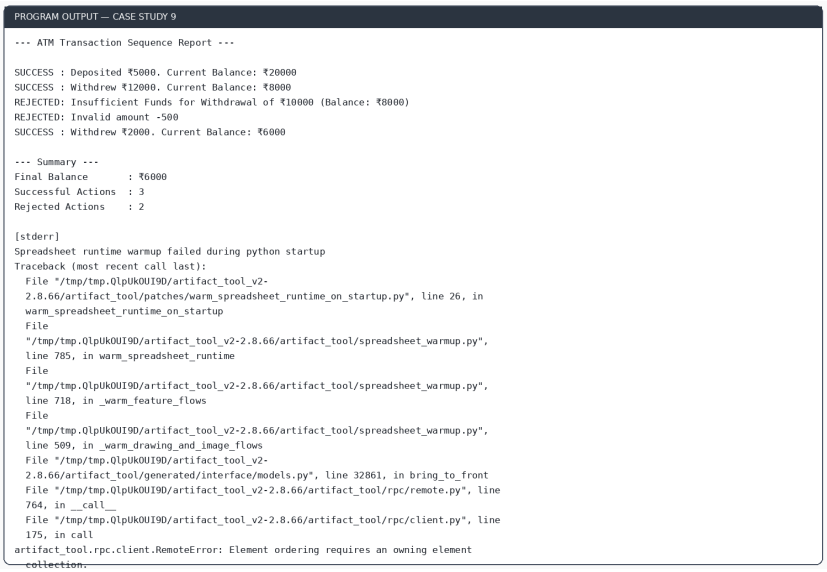

# Execute Solution
transactions = [("Deposit", 5000), ("Withdrawal", 12000), ("Withdrawal", 10000), ("Deposit", -500),
                ("Withdrawal", 2000)]
simulate_atm(15000, transactions)
```

8. Test Cases

Transaction	Expected result
Opening ₹15,000; Deposit ₹5,000	Accepted; balance ₹20,000.
Then Withdrawal ₹12,000	Accepted; balance ₹8,000; daily withdrawn ₹12,000.
Then Withdrawal ₹10,000	Rejected: insufficient funds (balance ₹8,000).
Deposit −₹500	Rejected: invalid amount.
Withdrawal ₹2,000	Accepted; final balance ₹6,000; 3 successful and 2 rejected actions.

9. Output Screenshot

The following screenshot-style capture was generated by executing the original uploaded Python file.



Execution return code: 0.

10. Conclusion

The simulation demonstrates sequential processing, tuple unpacking, validation and nested decision-making. It maintains a running balance and enforces a daily withdrawal cap.

CASE STUDY 10

Advanced Numeric Generator – Symmetric Hollow Diamond

1. Problem Statement

Generate a symmetric hollow numeric diamond for a positive odd integer. The upper and lower halves use increasing and decreasing odd numbers, with spaces creating a hollow center.

2. Objectives

• Validate that the input is a positive odd integer.

• Generate the upper half of the diamond using a loop.

• Generate the lower half in reverse order.

• Use spaces to align the sides and create the hollow interior.

3. Assumptions

• The input n is an integer.

• Only positive odd values are accepted.

• The sample execution uses n = 7.

• The pattern prints boundary numbers for each row rather than filling the interior.

4. Algorithm

1. Start and receive n.

2. If n is non-positive or even, display an error and stop.

3. Set leading spaces to n divided by two.

4. For odd values from 1 through n, build a row with leading spaces.

5. For the first row, print 1; otherwise print the row number, interior spaces and the same row number.

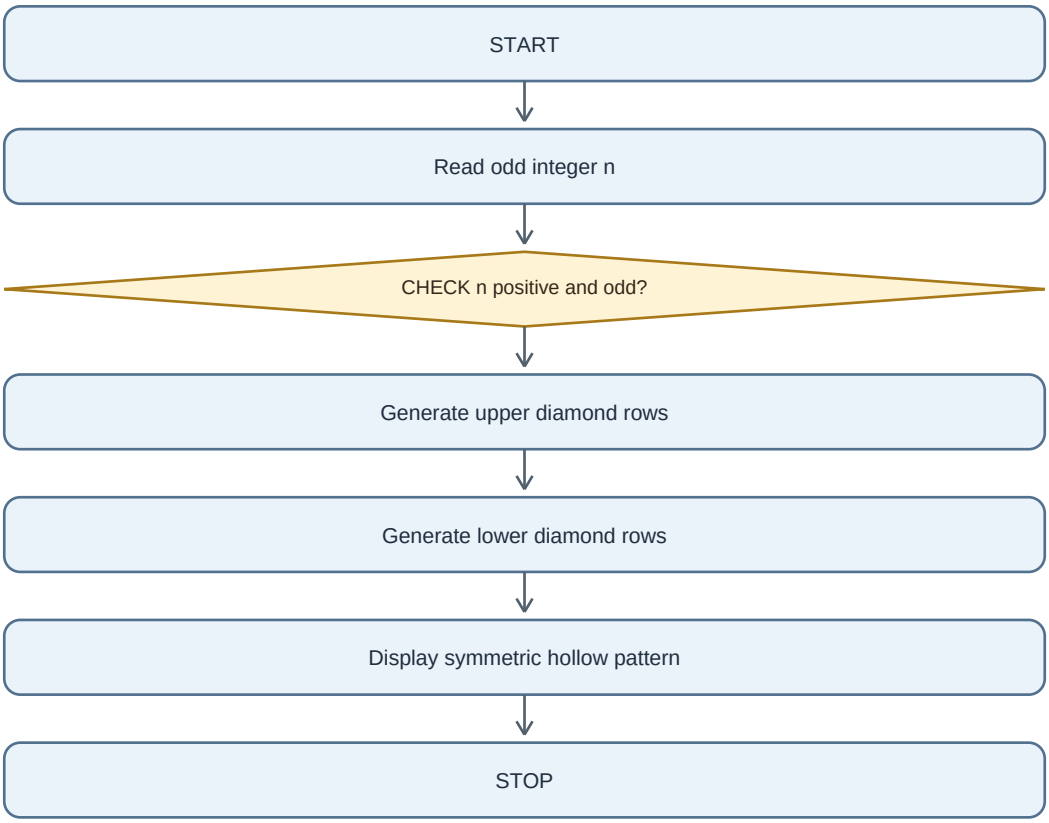
6. Print the row and reduce leading spaces by one.

7. Set lower-half leading spaces to one.

8. For odd values from n−2 down to 1, build and print rows using the same hollow format, increasing leading spaces.

9. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
INPUT n
IF n <= 0 OR n is even: DISPLAY validation message; STOP
spaces = n // 2
FOR i = 1, 3, 5, ..., n
  row = leading spaces
  IF i == 1: append '1'
  ELSE: append i + interior spaces + i
  DISPLAY row; spaces -= 1
END FOR
spaces = 1
FOR i = n-2, n-4, ..., 1
  BUILD and DISPLAY matching hollow row
  spaces += 1
END FOR
END
```

7. Python Source Code (as supplied)

```
def generate_numeric_diamond(n):
    if n <= 0 or n % 2 == 0:
        print("Please enter a positive odd integer.")
        return

    print(f"--- Symmetric Hollow Diamond Pattern (Size: {n}) ---\n")

    # Upper half of diamond
    spaces = n // 2
    for i in range(1, n + 1, 2):
        row_str = " " * spaces
        if i == 1:
            row_str += str(1)
        else:
            row_str += str(i) + " " * (2 * (i - 2) + 1) + str(i)
        print(row_str)
        spaces -= 1

    # Lower half of diamond
    spaces = 1
    for i in range(n - 2, 0, -2):
        row_str = " " * spaces
        if i == 1:
            row_str += str(1)
        else:
            row_str += str(i) + " " * (2 * (i - 2) + 1) + str(i)
        print(row_str)
```

```
        spaces += 1

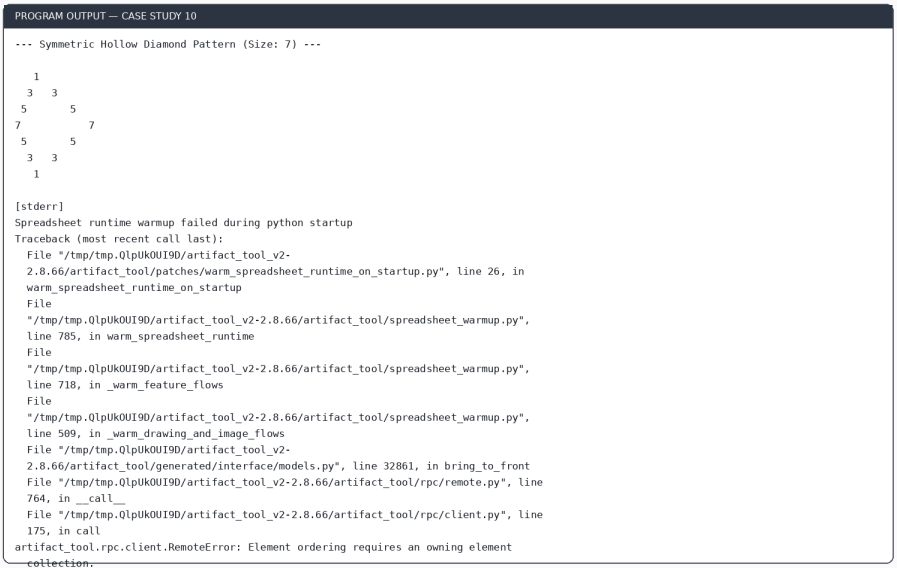

# Execute Solution
generate_numeric_diamond(7)
```

8. Test Cases

Input	Expected result
7	Prints a seven-row symmetric hollow diamond with outer values 1, 3, 5, 7, 5, 3, 1.
4	Displays: Please enter a positive odd integer.
0	Displays the same validation message.

9. Output Screenshot

The following screenshot-style capture was generated by executing the original uploaded Python file.



Execution return code: 0.

10. Conclusion

The pattern generator uses input validation, arithmetic, loops and string multiplication. Separate upper- and lower-half loops create a symmetric hollow diamond.